

LESSON 03 / STAGE ONE

Enumeration & Its Optimization

# 枚举算法与优化

暴力枚举 · 数学性质减层 · P7071 优秀的拆分精讲

把每一种可能都老老实实试一遍，再用数学性质把试不完的部分一刀砍掉——枚举，是"暴力"与"聪明"的合奏：范围卡得准，检验写得全，答案自然浮出水面。

03

授课班级

中预 2026 级 B 班

主讲

信息教研组

时间

2026 年 10 月

共 10 道课堂实践题 · 含洛谷题号

1

### 会列"所有可能"——枚举三要素

先想清楚三件事：枚举谁（对象）、从几到几（范围）、怎么验货（检验）。三要素一旦确定，代码骨架就定了——这是本课第一关键点。

2

### 会用数学性质砍枚举层

奇偶性先砍一半，整除性再砍一刀：P7071 里"奇数必无解"一条判断，一半数据瞬间出答案——本课题里"优化"二字的主角。

3

### 会用空间换时间——打表、前缀和、桶

把"第二层枚举"变成"查一次表"：A-B 数对查桶、火柴棒打表、刺痛值滑窗，10 万级数据也能瞬间出答案。

4

### 有复杂度意识——动手前先算账

循环套几层 × 每层多少次？估出来超过  $10^8$  就要换打法。"暴力"不是无脑，而是"算得清账的暴力"。

### 课堂安排

建议节奏，可按学情调整

15 分钟

概念讲解

25 分钟

例题精讲 P7071

55 分钟

上机实践（10 题）

10 分钟

总结答疑

### 承上启下

第 2 课我们学会了"照题意一步步做"；这一课把问题反过来——不告诉你怎么走，让你把所有走法都试一遍，再聪明地少试。

一句话定义

枚举，就是把问题所有可能的情况一个不落地列出来，逐个检验，把满足条件的留下来——朴素、完整、不会漏。

1 对象要选对

枚举"准"大有讲究。P1008 里枚举第一个三位数只要 207 个候选；老老实实枚举 9 个数字的全排列就是 36 万次——对象选得好，快上千倍。

2 范围要卡紧

范围每砍一半，时间就省一半。整除、奇偶、大小关系都是天然的"范围剪刀"——先用数学性质筛一遍，再动手写循环。

3 检验要写对

枚举出候选只是第一步，还要用 if 逐个验货。检验写漏一种情况（比如忘了 0 不能出现），整道题就从"对"变成"错"。

生活里的一次"枚举" —— 试密码箱（今天核心题的原型！）

拿到密码箱

000 ~ 999，一共 1000 种可能



逐个试

000、001、002……一个不跳



每次验货

转一下锁：开了没有？



得到答案

开了就是密码；全没开则不在范围

暴力，但保证不漏——这就是枚举的底气。剩下的功夫，全在"怎么少试几把"。

C++ 视角

一至多重循环 + if 检验，就是枚举的全部骨架。CSP-J 入门级 T1 常客：数据范围往往刚好"逼你"先算复杂度——把范围卡准、检验写全，就是最稳的得分手段。

# 关键点一：枚举三要素——对象、范围、检验

## 三要素 checklist：写枚举前先问自己三个问题

- 1

**对象：枚举谁？**

找"定一个、牵全部"的量。三连击里定下第一个数  $x$ ， $y = 2x$ 、 $z = 3x$  全跟着定了——枚举 1 个变量，而不是 9 个数字的全排列（36 万种）。
- 2

**范围：从几到几？**

范围是约束"推"出来的：1~9 不重不漏  $\rightarrow x$  最小 123； $z = 3x$  必须是三位数  $\rightarrow x \leq 987 \div 3 = 329$ 。两刀砍完只剩 207 个候选。
- 3

**检验：怎么算对？**

cnt[10] 桶逐位计数： $x$ 、 $y$ 、 $z$  的 9 个数字，1~9 各恰好出现一次、0 不出现才算过关。检验是枚举的"验货环节"，一种情况都不能漏。

### 一句话记住

对象、范围、检验三票齐了再动手——三要素里有两票（对象、范围）都花在"少枚举"上，优化的功夫在写代码之前。

## 主循环伪代码：三要素各就各位

```
for x = 123 .. 329:           // ② 范围 (① 对象是 x)
    y = 2 * x;  z = 3 * x
    cnt[0..9] 全部清零
    把 x、y、z 的 9 个数字
    逐位丢进 cnt[] 计数
    ok = (1..9 各恰好一次)
    且 cnt[0] == 0
    if ok: 输出 x y z         // ③ 检验
```

三个约束（三位数、1 : 2 : 3、数字 1~9 不重不漏）用掉两个来卡范围，只剩一个留给 if——这是枚举优化的常用套路。

### 常见错位

范围拍脑袋开成 100~999：900 组候选大半是废票，还可能把数字 0 混进 cnt 里忘了排除。范围不是试出来的，是用约束"推"出来的。

# 关键点二：用数学性质砍枚举层

## 一道真题：把 n 拆成互不相同的 2 的正整数次幂之和

6 = 4 + 2 是优秀的拆分；8 = 4 + 4 不行（4 重复）；10 = 7 + 3 也不行（7、3 不是 2 的幂）。n ≤ 10<sup>7</sup> 时 2<sup>24</sup> 已超过它——暴力做法：24 个幂每个"选或不选"，2<sup>24</sup> ≈ 1700 万种子集，1 秒时限擦线赌博。

### 性质① 奇数必无解

2、4、8……全是偶数，偶数之和还是偶数。n 是奇数 → 直接输出 -1：一半的数据瞬间判掉，循环都省了。

### 性质② 相同幂可合并

2<sup>k</sup> + 2<sup>k</sup> = 2<sup>k+1</sup>，所以最优拆法里每个幂最多出现一次 → 从大到小贪心"能拿就拿"，n = 10<sup>7</sup> 也只跑 24 轮。

还要砍第三刀：贪心为什么对？从大到小"能拿就拿"，剩下的零头刚好被小幂补齐——选出来的组合恰好就是 n 的二进制表示，天然互不相同，方案唯一。

### 本课反复出现的思维动作

先读懂数据的"形状"（奇偶、整除、单调），再决定循环怎么写——性质就是剪刀。

## 主循环伪代码：两条性质各占一行

```
读入 n

if n % 2 == 1:                // 性质① 奇数无解
    输出 -1; 结束

for i = 24 down to 1:        // 2^24 > 10^7
    if n ≥ 2^i:
        选下 2^i;  n -= 2^i

输出选下的所有幂

(顺序自然从大到小)
```

注意输出顺序：从大到小枚举，选下的幂天然从大到小排列——与题目要求的输出顺序一致，连排序都省了。

### 效果对照

O(2<sup>24</sup>) ≈ 1.7×10<sup>7</sup> 子集枚举 → O(24) 贪心：一条数学性质砍掉约 70 万倍的枚举量——这就是本标题里"优化"二字的分量。

# 关键点三：打表、前缀和、桶——空间换时间

## 三件"省力气"的武器

### A 预处理打表（T05 火柴棒）

先扫一遍算出  $need[i] = \text{数 } i \text{ 需要几根火柴}$ ，枚举  $a、b$  时查表  $O(1)$ 。不做预处理，每个候选都要重新数一遍位。

### B 桶计数（T04 A-B 数对）

$A - B = C$  变形为  $B = A - C$ 。每个数出现几次先数进桶，枚举  $A$  时"查一次桶"代替"再扫一遍数组"—— $O(n^2)$  变  $O(n)$ 。

### C 前缀和 / 滑动窗口（T03 刺痛值）

连续  $m$  个数的最小和：窗口右进左出只做一次加减，不用每个窗口从头加—— $O(n \cdot m)$  变  $O(n)$ 。

#### 共同的本质

枚举优化的三板斧，本质都是"把重复算过的东西存起来"：数过的位数存成表、数过的个数存进桶、加过的和留在窗口里——用一点空间，换大量时间。

## 复杂度对照账（本课三道代表题）

| 题目           | 朴素枚举                         | 优化后                        |
|--------------|------------------------------|----------------------------|
| P1149 火柴棒等式  | 每个候选重新数位                     | 打表后 $1.2 \times 10^6$ 次查表  |
| P1102 A-B 数对 | 双层 for $4 \times 10^{10}$ 必炸 | 查桶 $O(n) = 2 \times 10^5$  |
| P1614 爱与愁的心痛 | 每窗从头加 $6 \times 10^{10}$     | 滑动窗 $O(n) = 2 \times 10^5$ |

#### 算账标准

朴素做法一旦超过  $10^8$  次基本运算，1 秒时限基本告吹；优化后  $10^6$  量级"闭眼过"。动手前先列这张表，再决定写不写优化。

#### 常见误区

不是所有题都要优化： $n \leq 1000$  的双重循环才  $10^6$  次，朴素枚举完全够用——先算账再动手，别为了优化而优化。

# 本课十题总览

| 编号  | 题目           | 来源            | 难度  | 枚举对象与优化手法                               |
|-----|--------------|---------------|-----|-----------------------------------------|
| T01 | 优秀的拆分 P7071  | CSP-J 2020 T1 | ★★★ | 从大到小枚举 2 的幂：奇偶性砍半，24 轮贪心                |
| T02 | 三连击 P1008    | NOIP1998 普及组  | ★★  | 枚举第一个数 $x \in [123, 329]$ ：约束反推范围       |
| T03 | 爱与愁的心痛 P1614 | 洛谷经典          | ★★  | 枚举窗口：滑窗右进左出， $O(n \cdot m)$ 变 $O(n)$    |
| T04 | A-B 数对 P1102 | 洛谷经典          | ★★★ | 枚举 A 查桶： $B = A - C$ ，查表代替第二层           |
| T05 | 火柴棒等式 P1149  | NOIP2008 提高组  | ★★★ | 枚举 a、b：need[] 打表，数位不重算                  |
| T06 | 选数 P1036     | NOIP2002 普及组  | ★★★ | 组合枚举（选 / 不选）：试除到 $\sqrt{\text{sum}}$ 判素 |
| T07 | 烤鸡 P2089     | 洛谷经典          | ★★  | 十种配料全枚举：可行性剪枝砍路径                        |
| T08 | 扫雷游戏 P2670   | NOIP2015 普及组  | ★★  | 枚举格子 $\times 8$ 方向：方向数组免写 8 个 if        |
| T09 | 统计天数 P1567   | 洛谷经典          | ★   | 线性扫描：一段一段数，一遍跑完                         |
| T10 | 欢乐的跳 P1152   | 洛谷经典          | ★★  | 枚举相邻差：桶标记验「1..n-1 齐全」                   |

全部 10 题的参考代码均已通过洛谷官方样例验证。课堂节奏：T01 精讲 5 页；T02 – T10 每题 4 页（题面 → 思路 → 方法 → 代码），可按学情取舍。难度为课堂参考值，均不超过 CSP-J T1 水平。

# 精讲 T01 | 优秀的拆分（P7071）

## 题面描述

一般来说，一个正整数可以拆分成若干个正整数的和。我们称一种拆分是「优秀的」，当且仅当  $n$  被分解成了若干个互不相同的  $2$  的正整数次幂之和。

**例如：** $6 = 4 + 2$  是优秀的拆分；

**反例：** $8 = 4 + 4$  不行（ $4$  重复了）； $10 = 7 + 3$  也不行（ $7$ 、 $3$  不是  $2$  的幂）。

给定正整数  $n$ ，判断它是否存在优秀的拆分；若存在，给出具体的拆分方案。

$2$  的正整数次幂： $2$ 、 $4$ 、 $8$ 、 $16$ ……（ $2^0 = 1$  不算—— $0$  不是正整数！）

题目保证：方案存在时，拆分方案是唯一的。

## 思路预告

别急着写循环——先把「 $2$  的正整数次幂」这个限制翻译成数学性质，答案会自己跳出来。第 5 页已经预告了两条性质，这里正式派上用场。

## 输入格式

输入只有一行，一个整数  $n$ ，代表需要拆分的数。

## 输出格式

无解输出  $-1$ ；有解则从大到小输出拆分中的每个数，相邻两数用空格隔开。

## 数据范围

$1 \leq n \leq 10^7$ （ $2^{24} = 16777216$  已超过它）。

## 三组官方样例

| 输入  | 输出             |
|-----|----------------|
| 6   | 4 2            |
| 7   | -1             |
| 126 | 64 32 16 8 4 2 |

样例 3 的一行里藏着 6 个互不相同的  $2$  的幂——把  $126$  的二进制写出来看看？

# 题面 → 代码：暴力为何险，贪心为何稳

## 主循环伪代码：一条性质一行

```
读入 n

if n % 2 == 1:  输出 -1; 结束      // 性质① 奇数无解

for i = 24 down to 1:              // 2^24 > 10^7
    if n ≥ 2^i:
        选下 2^i;  n -= 2^i      // 能拿就拿

输出选下的所有幂（自然从大到小）
```

暴力子集枚举要  $2^{24} \approx 1.7 \times 10^7$  次判断——能过但纯属赌博；贪心只跑 24 轮，稳稳收工。

贪心选出的幂集合恰好就是 n 的二进制表示里那些 1 对应的位权——由于 n 是偶数，二进制末位必为 0，所以不会出现  $2^0$ ，正整数次幂的约束自动满足。

### Q1 · 为什么从大到小？

每个幂最多选一次（性质②），大幂先占坑，剩下的零头刚好被小幂补齐——这正是 n 的二进制表示，方案唯一。

### Q2 · 为什么奇数直接 -1？

2、4、8……全是偶数，偶数怎么加都是偶数；奇数无论怎么拆都无解。一半数据瞬间判掉。

### Q3 · 为什么不会剩下 1？

n 是偶数才进循环；从大到小拆到最后剩下的也是偶数，最小剩 2—— $2^0$  从头到尾没机会出现。

### 一句话

奇数砍一半、互不相同砍分支——两条性质一上，24 轮循环稳稳收工，这就是「枚举 + 优化」的完整闭环。

状态设计 + 样例手推：126 → 64 32 16 8 4 2

- 1

n: 还没拆完的剩余部分——每选一个幂就减一笔。
- 2

ans 序列: 选下的幂, 按从大到小输出 (枚举顺序天然有序)。  
状态清单: 程序里要维护什么
- 3

循环变量 i: 当前尝试的幂次, 从 24 到 1。

| 轮次 i | $2^i$    | n (判断前) | $n \geq 2^i ?$ | 动作 / 剩余 n   |
|------|----------|---------|----------------|-------------|
| 24   | 16777216 | 126     | 否              | 跳过 / 126    |
| 6    | 64       | 126     | 是              | 选 64 / 剩 62 |
| 5    | 32       | 62      | 是              | 选 32 / 剩 30 |
| 4    | 16       | 30      | 是              | 选 16 / 剩 14 |
| 3    | 8        | 14      | 是              | 选 8 / 剩 6   |
| 2    | 4        | 6       | 是              | 选 4 / 剩 2   |
| 1    | 2        | 2       | 是              | 选 2 / 剩 0 ✓ |

两条对照线

① 输出 64 32 16 8 4 2 = 官方样例 3, 从大到小一行搞定; ② 若  $n = 7$ : 第一行性质①就直接输出 -1, 连循环都不进。第 1、2 组样例由此全覆盖。

手推 = 人肉当 CPU

左列写状态, 右表写动作, 一行一行走——写代码前先在纸上通一遍样例。

# 参考代码：24 轮贪心，一气呵成

```
#include <cstdio>
int main() {
    long long n;
    scanf("%lld", &n);
    if (n % 2 == 1) {           // 性质① 奇数必无解
        printf("-1
");
        return 0;
    }
    bool first = true;          // 控制空格输出
    for (int i = 24; i >= 1; i--) { // 2^24 > 10^7
        long long p = 1LL << i;
        if (n >= p) {           // 能拿就拿：贪心
            n -= p;
            if (!first) printf(" ");
            printf("%lld", p);
            first = false;
        }
    }
    printf("
");
    return 0;
}
```

### 结构一览

没有 struct、没有数组——一个 n、一层 for、一个 if，全题完事。唯一要记住的是  $2^{24} > 10^7$ ，循环上界取 24。

### 三步对号入座

if (n % 2) 行 = 性质①奇偶判断；for 行 = 性质②从大到小；if (n ≥ p) 行 = 「能拿就拿」贪心。与第 9 页伪代码一一对应。

### 两个好习惯

① 2 的幂用 1LL 左移 i 位写，long long 不越界；② 用 first 标记控制空格，行尾不多打空格——很多 OJ 会判 Presentation Error。

### 提交提示

课后亲手敲一遍并提交洛谷 P7071。三组官方样例 (6→4 2; 7→-1; 126→64 32 16 8 4 2) 及边界值 1、2、 $10^7$  均已通过验证脚本验证。

错1 忘判奇数，硬拆到底

$n = 7$  若硬跑循环，最后会剩 1 拆不掉——2 的正整数次幂里没有 1。输出残缺或错判有解，先判奇数一步到位。

错2 把  $2^0 = 1$  也当成拆分项

$1 = 2^0$ ，但 0 不是正整数！循环从  $i = 24$  到 1，天然避开  $2^0$ ；如果自己写 `while(n > 0)` 一路减，就会踩坑。

错3 2 的幂用 int 硬算

1 左移 24 位装得下 int，但顺手写成  $2^i$  逐次相乘或取错位极易溢出；统一用 1LL 左移写幂， $n$  又是 long long，一步到位。

复杂度小账

$O(24)$

暴力子集枚举  $2^{24} \approx 1.7 \times 10^7$ ，1 秒时限属于「擦线赌博」——机器快一点慢一点全看命。贪心只跑 24 轮， $n = 10^7$  也是瞬间完成。这就是数学性质的分量：把「碰运气」变成「必然」。

本讲要点回收

奇偶判 → 从大到小 → 能拿就拿。十二个字，就是这道 CSP-J T1 的全部。

题面描述

将 1 ~ 9 共 9 个数字分成 3 组，分别组成 3 个三位数，且使这 3 个三位数构成 1 : 2 : 3 的比例。试求出所有满足条件的 3 个三位数。

本题没有输入——答案固定，直接把所有解按每行第一个数升序输出即可。

**三个约束：**① 每个数都是三位数；② 三数之比 1 : 2 : 3；③ 数字 1~9 在三个数中不重不漏地各出现一次。

**反例：**100 200 300（有 0，且数字重复）；123 246 369（数字重复）。

输入格式

无输入。

输出格式

若干行，每行 3 个数，按每行第 1 个数升序排列。

数据范围

无（本题直接输出全部答案）。

官方样例

| 输入 | 输出                                                                                   |
|----|--------------------------------------------------------------------------------------|
| 无  | <b>192 384 576</b><br><b>219 438 657</b><br><b>273 546 819</b><br><b>327 654 981</b> |

共 4 行，全部答案——课后可自行核对。

思路预告

枚举对象选谁，效率天差地别：定下第一个数，另外两个自动确定——这个"牵一发动全身"的量就是最佳枚举对象（第 4 页的关键点一）。

# 思路梳理：枚举一个，牵动三个

### 三要素逐项落实

#### 对象：枚举谁？

$x$  定了,  $y = 2x$ 、 $z = 3x$  全跟着定了——枚举 1 个变量, 而不是 9 个数字的全排列 (36 万种)。

#### 范围：从几到几？

$z = 3x$  必须是三位数  $\rightarrow x \leq 987 \div 3 = 329$ ;  $x \geq 123$  (1~9 不重不漏的最小排布)。207 个候选, 快上千倍。

#### 检验：查什么？

$x$ 、 $y$ 、 $z$  共 9 个数字逐位计数: 1~9 各恰好 1 次、0 一次都不出现。 $\text{cnt}[10]$  桶数组一把搞定。

### 主循环伪代码

```
for x = 123 .. 329:           // ② 范围
    y = 2x;  z = 3x
    cnt[0..9] 清零
    拆出 x、y、z 的每个数字
    数字 d:  cnt[d]++
    ok = (cnt[0] == 0) 且
           1..9 各恰好 1 次
    if ok: 输出 x  y  z      // ③ 检验
```

### 一句话记住

三个约束用掉两个卡范围, 只剩一个留给 if——"范围是推出来的, 不是拍出来的", 第 4 页的原则在这里落地。

# 方法拆解 + 样例手推：x = 192 走一遍

## 方法三步

### ① 定对象

$x = 192$ :  $y = 2x = 384$ 、 $z = 3x = 576$ ，三位数与  $1 : 2 : 3$  自动满足。

### ② 拆数字

1 9 2 3 8 4 5 7 6 逐位丢进 cnt——9 个格子各记一笔。

### ③ 查桶

$\text{cnt}[0] = 0$  ✓; 1~9 各恰好 1 次 ✓ → 输出 192 384 576。

### 顺序白送

答案恰好按  $x$  升序——枚举顺序天然有序，连排序都省了。

| 步骤          | 操作                      | 结果            |
|-------------|-------------------------|---------------|
| 定 $x = 192$ | 算 $y = 384$ 、 $z = 576$ | 1:2:3 ✓ 三位数 ✓ |
| 拆数字计数       | 1,9,2,3,8,4,5,7,6 逐位 ++ | cnt 共 9 笔     |
| 查 0         | $\text{cnt}[0] == 0$    | ✓             |
| 查 1..9      | 各恰好 1 次                 | ✓ 输出          |

### 全部 4 组解

192 384 576、219 438 657、273 546 819、327 654 981——已用验证脚本对过官方输出，与本页手推同构。

```
#include <cstdio>
int main() {
    for (int x = 123; x <= 329; x++) {    // 对象+范围
        int y = 2 * x, z = 3 * x;
        int cnt[10] = {0};                // 循环内定义,自动清零
        int t[3] = {x, y, z};
        for (int i = 0; i < 3; i++)
            for (int v = t[i]; v > 0; v /= 10)
                cnt[v % 10]++;            // 逐位计数
        bool ok = (cnt[0] == 0);
        for (int d = 1; d <= 9; d++)
            if (cnt[d] != 1) ok = false;    // 1~9 各恰好一次
        if (ok) printf("%d %d %d\n", x, y, z);
    }
    return 0;
}
```

结构一览

双层 for (外枚举 x、内拆数字) + cnt 桶 + 一轮检查——纯数组无 STL，初学者友好。cnt 定义在循环内，每轮自动清零。

三步对号入座

for x 行 = 对象 + 范围；内层两行 = 把候选"摊开"计数；if (ok) = 检验输出。与第 14 页伪代码一一对应。

两个好习惯

① cnt 定义在循环体内自动清零，不会让上一轮的脏数据串场；② 输出行末直接换行，格式干净。

提交提示

课后亲手敲一遍并提交洛谷 P1008。输出共 4 行，已通过验证脚本对拍官方答案。

# T03 | 爱与愁的心痛（P1614）

## 题面描述

最近有  $n$  件不爽的事，每件有一个刺痛值。爱与愁大神想知道：连续  $m$  件刺痛值的和最小是多少？

「连续」是关键词——只能取相邻的一段，不能挑挑拣拣。

本课定位：滑动窗口的入门题——"前缀和 / 滑窗"这把武器的第一次实战。

## 思路预告

暴力每个窗口从头加要  $O(n \cdot m) \approx 6 \times 10^{10}$ ，必炸——相邻窗口共享  $m-1$  个数，让窗口"滑"起来。

## 输入格式

第一行两个整数  $n\ m$ ；接下来  $n$  行，每行一个整数  $a_i$ ，代表第  $i$  件事的刺痛值。

## 输出格式

一行一个整数：连续  $m$  件刺痛值的和的最小值。

## 数据范围

$1 \leq m \leq n \leq 2 \times 10^5$ 。

## 官方样例

| 输入                                     | 输出 |
|----------------------------------------|----|
| 8 3<br>1<br>4<br>7<br>3<br>1<br>2<br>4 | 6  |
| 答案是 $3 + 1 + 2 = 6$ ——连续第 4~6 件事。      |    |
| 3                                      |    |

从暴力到滑窗

暴力差在哪？

枚举每个左端点，从  $a[i]$  加到  $a[i+m-1]$ ——相邻两个窗口重复加了  $m-1$  个数。重复劳动就是优化空间。

滑窗怎么做？

先加好前  $m$  个数得  $s$ ； $i$  从  $m$  到  $n-1$ ：  $s += a[i] - a[i-m]$ ，右进一个、左出一个，每步  $O(1)$ ，总共  $O(n)$ 。

为什么是对的？

每个窗口恰好被算一次，共享部分进出抵消——数学上就是"前缀和之差"，第 9 课递推还会再见到它。

主循环伪代码

```
读入 n, m 和数组 a

s = a[0]+...+a[m-1]    // 第一扇窗

best = s

for i = m .. n-1:

    s += a[i] - a[i-m]    // 右进左出

    best = min(best, s)

输出 best
```

复杂度小账

$O(n \cdot m) \approx 6 \times 10^{10} \rightarrow O(n) = 2 \times 10^5$ ：窗口和最大可达  $10^8$  量级——int 擦线，long long 稳妥；先算账再选类型，老规矩。

# 方法拆解 + 样例手推：答案是 6

### 滑窗三步

#### ① 第一扇窗

$a[0..2] = 1 + 4 + 7 = 12$ ,  $best = 12$ 。

#### ② 滑动 5 步

每步一加一减： $12 \rightarrow 14 \rightarrow 11 \rightarrow 6 \star \rightarrow 7 \rightarrow 9$ ,  $best$  一路刷新。

#### ③ 输出

$best = 6 =$  官方样例答案  $\checkmark$ 。

#### 重复利用

窗口共  $n-m+1$  个；"重复利用已经算过的和"是枚举优化的第一直觉。

| 窗口（连续 3 件） | 进 / 出 | 窗和        | best |
|------------|-------|-----------|------|
| [1, 4, 7]  | 初始化   | 12        | 12   |
| [4, 7, 3]  | +3 -1 | 14        | 12   |
| [7, 3, 1]  | +1 -4 | 11        | 11   |
| [3, 1, 2]  | +2 -7 | 6 $\star$ | 6    |
| [1, 2, 4]  | +4 -3 | 7         | 6    |
| [2, 4, 3]  | +3 -1 | 9         | 6    |

#### 手推结论

6 步滑完， $best = 6$ ，与官方样例一致——手推一遍，滑窗的正确性肉眼可见。

# 参考代码：右进左出

```
#include <iostream>
#include <algorithm>
using namespace std;
int a[200005];
int main() {
    int n, m;
    cin >> n >> m;
    for (int i = 0; i < n; i++) cin >> a[i];
    long long sum = 0;
    for (int i = 0; i < m; i++) sum += a[i]; // 第一扇窗
    long long best = sum;
    for (int i = m; i < n; i++) {
        sum += a[i] - a[i - m]; // 右进左出 O(1)
        best = min(best, sum);
    }
    cout << best << endl;
    return 0;
}
```

## 结构一览

一个数组、两个 long long (sum、best)、两层循环——没有套路的"套路题"，重点全在"右进左出"这一行。

## 三步对号入座

第一个 for = 初始化第一扇窗；第二个 for = 滑动；min = 验货。与第 18 页伪代码逐行对应。

## 两个好习惯

① 窗口和用 long long 防溢出；② 20 万级数据可加 ios::sync\_with\_stdio(false) 加速读入。

## 提交提示

课后亲手敲一遍并提交洛谷 P1614。官方样例 8 3 → 6 已通过验证脚本验证。

题面描述

给出一串正整数数列和一个正整数 C，要计算出所有满足  $A - B = C$  的数对的个数。

**注意：**不同位置的数字一样的数对，算不同的数对——1 1 里两个 1 互相配，位置不同就算两对。

本课定位：桶计数的入门题——"查表代替第二层枚举"的完整示范。

思路预告

双层 for 枚举所有数对是  $O(n^2) = 4 \times 10^{10}$ ，必炸——把等式变形，第二层就能"查表"。

输入格式

共两行：第一行两个正整数 n c；第二行 n 个正整数  $a_i$ 。

输出格式

一行，满足  $A - B = C$  的数对的个数。

数据范围

$1 \leq n \leq 2 \times 10^5$ ；数值可达  $10^9$ （变量用 long long，桶用 map）。

官方样例

输入

4 1  
1 1 2 3

输出

3

三对：(2,1)、(2,1)'（两个 1 各配一次）、(3,2)。

# 思路梳理：变形 + 查桶

## 两步走

### 第一步：变形

$A - B = C$  等价于  $B = A - C$ 。枚举每个  $A$ ，答案加上「 $A - C$  出现了几次」——配对变成了查询。

### 第二步：查表

每个数出现几次，先一次性数进 map 桶 cnt；查询一次  $O(\log n)$ 。总复杂度  $O(n \log n) \approx 3 \times 10^6$ 。

### 第三步：防坑

$C$  是正数  $\rightarrow A \neq B$ ，不会自己配自己；"位置不同算不同对"  $\rightarrow$  桶里存「次数」而不是「有没有」，累加自然算上重复。

## 主循环伪代码

```
读入 n, c
for i = 1 .. n:
    读入 a[i]
    cnt[a[i]]++           // 数进桶
ans = 0
for i = 1 .. n:
    ans += cnt[a[i]-c] // 枚举A查B
输出 ans
```

## 一句话记住

正难则反：与其枚举"谁配谁"，不如枚举一个、查另一个——变形公式把主动权交回给你。

# 方法拆解 + 样例手推：答案是 3

## 手推两步

### ① 数桶

数列 1 1 2 3,  $C = 1 \rightarrow \text{cnt: } 1 \rightarrow 2 \text{ 次、} 2 \rightarrow 1 \text{ 次、} 3 \rightarrow 1 \text{ 次。}$

### ② 查桶

$A=1$ : 查  $B=0 \rightarrow 0 \text{ 次}$ ;  $A=2$ : 查  $B=1 \rightarrow 2 \text{ 次★}$ ;  $A=3$ : 查  $B=2 \rightarrow 1 \text{ 次★}$ 。

### ③ 合计

$0 + 0 + 2 + 1 = 3$  ✓ 官方样例答案。

### 次数即答案

同一个数出现几次，查询时就贡献几次——桶存「次数」的设计在这里见效。

| 枚举 A | 查 $B = A - C$ | 桶值  | 累计 |
|------|---------------|-----|----|
| 1    | 0             | 0   | 0  |
| 1    | 0             | 0   | 0  |
| 2    | 1             | 2 ★ | 2  |
| 3    | 2             | 1 ★ | 3  |

### 手推结论

答案是 3，与官方样例一致。排序 + 二分也能做——但桶更直观，第 20 课优先队列还会跟它打交道。

# 参考代码：map 当桶

```
#include <iostream>
#include <map>
using namespace std;
map<long long, long long> cnt; // 数值可达1e9
long long a[200005];
int main() {
    int n; long long c;
    cin >> n >> c;
    for (int i = 0; i < n; i++) {
        cin >> a[i];
        cnt[a[i]]++;           // 先数进桶
    }
    long long ans = 0;
    for (int i = 0; i < n; i++)
        ans += cnt[a[i] - c];  // 枚举A查B
    cout << ans << endl;
    return 0;
}
```

## 结构一览

map 容器当桶（键和值都开 long long）：数值可达  $10^9$ ，普通数组开不下下标——数据范围决定容器，这是本页最重要的一句。

## 三步对号入座

第一个 for = 读入 + 数桶；第二个 for = 枚举 A 查 B。两行核心代码，与第 22 页伪代码一致。

## 两个好习惯

① A、C、桶全部 long long；② 先数完桶再查询，顺序不能反——边数边查会漏配对。

## 提交提示

课后亲手敲一遍并提交洛谷 P1102。官方样例 4 1 → 3 已通过验证脚本验证。